How To Create Tests

Tutorial

Setup

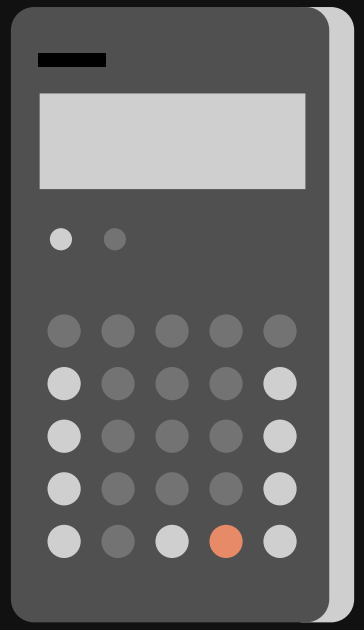
- Decide on a programming language (python, java, etc.)
- Install a testing framework for unit tests (pytest, jest, etc.)
- Why use a testing framework?
  - Simplifies the creation, execution and maintenance of tests
  - Built-in reporting tools that generate detailed summaries
  - Easily integrated into CI/CD pipelines

Functional Requirements

- Arithmetic Calculations:
  - Perform basic arithmetic functions:
    - Perform addition
    - Perform subtraction
    - Perform multiplication
    - Perform division
- Input Handling:
  - Accept input
  - Input must be 1 or 2 digits
  - Input must contain valid hexadecimal numbers
  - Input must include a valid operator (+, -, /, *)
- Output Handling:
  - Return and display output
  - Output must be less than 5 digits (<=4)
  - Output must not be negative
  - Output must not contain decimal places
- Error Handling:
  - Invalid hexadecimal input must return an error
  - Missing or invalid operator must return an error
  - Input exceeding 2 digits must return an error
  - Division by zero must return an error
    Output that is greater than 4 digits will return an error
  - Output that is negative will return an error
  - Output with decimal places will return an error

Steps for each test case

- Arrange
  - Setup the inputs needed
- Act
  - Call the function being tested
- Assert
  - Check the result is what you expect



Testing: Arithmetic Calculations

- Perform basic arithmetic functions:
  - Perform addition
  - Perform subtraction
  - Perform multiplication
  - Perform division

Testing: Arithmetic Calculations

- Arrange

```
def test_valid_addition_simple():
    res = perform_addition("1", "1")
    assert res ==  "2"
```

Testing: Arithmetic Calculations

- Act

```
def test_valid_addition_simple():
    res = perform_addition("1", "1")
    assert res ==  "2"
```

Testing: Arithmetic Calculations

- Assert

```
def test_valid_addition_simple():
    res = perform_addition("1", "1")
    assert res ==  "2"
```

Testing: Arithmetic Calculations

```python
def test_valid_addition_complex():
    res = perform_addition("8A", "B7")
    assert res ==  "141"
```

Testing: Arithmetic Calculations

```python
def test_valid_addition_simple():
    res = perform_addition("1", "1")
    assert res ==  "2"
```

```python
def test_valid_addition_complex():
    res = perform_addition("8A", "B7")
    assert res ==  "141"
```

Testing: Input Handling

- Accept input
- Input must be 1 or 2 digits
- Input must contain valid hexadecimal numbers
- Input must include a valid operator (+, -, /, *)

```python
def test_contains_valid_operator():
    res = contains_valid_operator("+")
    assert res == True

def test_does_not_contain_operator():
    res = contains_valid_operator("=")
    assert res == False
```

Testing: Output Handling

- Return and display output
- Output must be less than 5 digits (<=4)
- Output must not be negative
- Output must not contain decimal places

```python
def test_is_valid_digit_amount():
    res = handle_output("1F")
    assert res == "1F"

def test_is_positive():
    res = handle_output("1")
    assert res == "1"

def test_contains_decimal_places():
    res = handle_output("1")
    assert res == "1"
```

Testing: Error Handling

- Invalid hexadecimal input must return an error
- Missing or invalid operator must return an error
- Input exceeding 2 digits must return an error
- Division by zero must return an error
  Output that is greater than 4 digits will return an error
- Output that is negative will return an error
- Output with decimal places will return an error

```python
def test_handle_invalid_input():
    with pytest.raises(ValueError) as error:
        handle_input("1F +")
    assert str(error.value) == "Input must have atleast 2 hexadecimals values and 1 operator"
```

Next Step

Drafting the code